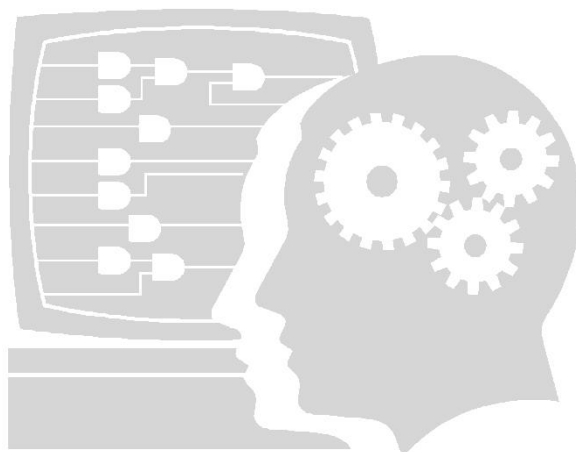


TD软件实验

安路 EG4S20

第一版



修改日期：2024 年 10 月

杭州康芯电子有限公司

www.kx-soc.com

目 录

前言.....	1
第一章 利用Anlogic TD 完成工程的基本流程.....	2
1.1 新建工程.....	3
1.2 输入设计.....	4
1.3 编译工程.....	7
1.4 仿真工程.....	7
1.5 分配管脚.....	7
1.6 编译布线.....	10
1.7 下载程序.....	11
1.8 固化程序.....	13
第二章 ModelSim 工程实战之手动仿真.....	14
2.1 编写 TestBench 仿真文件.....	14
2.2 配置仿真工程.....	15
2.3 开始工能仿真.....	16

第1章 利用 Anlogic TD 完成工程的基本流程

在本文中你将能够学到：

- 1 在Anlogic 中新建工程，编译工程的基本方法。
- 2 在Anlogic 中锁定引脚及制作引脚基本方法。
- 3 在Anlogic 中下载工程至开发系统的基本方法。
- 4 在这篇教程中，笔者将以一个简单的 LED 闪烁灯为例子，从工程实战的角度，带大家熟悉基于 Anlogic 从设计到实现下载到开发系统 最基本的使用方法。
- 5 编程及固化方法。

§ 1 Anlogic TD 工程实战

1.1 新建工程

下面我们就来创建一个新工程。在创建工程之前，我们建议大家硬盘中专门建立一个文件夹专用于存储自己的 Anlogic 工程，这个工程目录的路径名应该只有字母、数字和下划线，以字母为首字符，且不要包含中文和其他符号。创建好了专属文件夹后，我们便可以打开我们的开发工具，这里我们使用的开发工具是Anlogic 标准版(TD5.6.2-64bit)打开Anlogic 软件，软件主界面如图 1-0 所示。

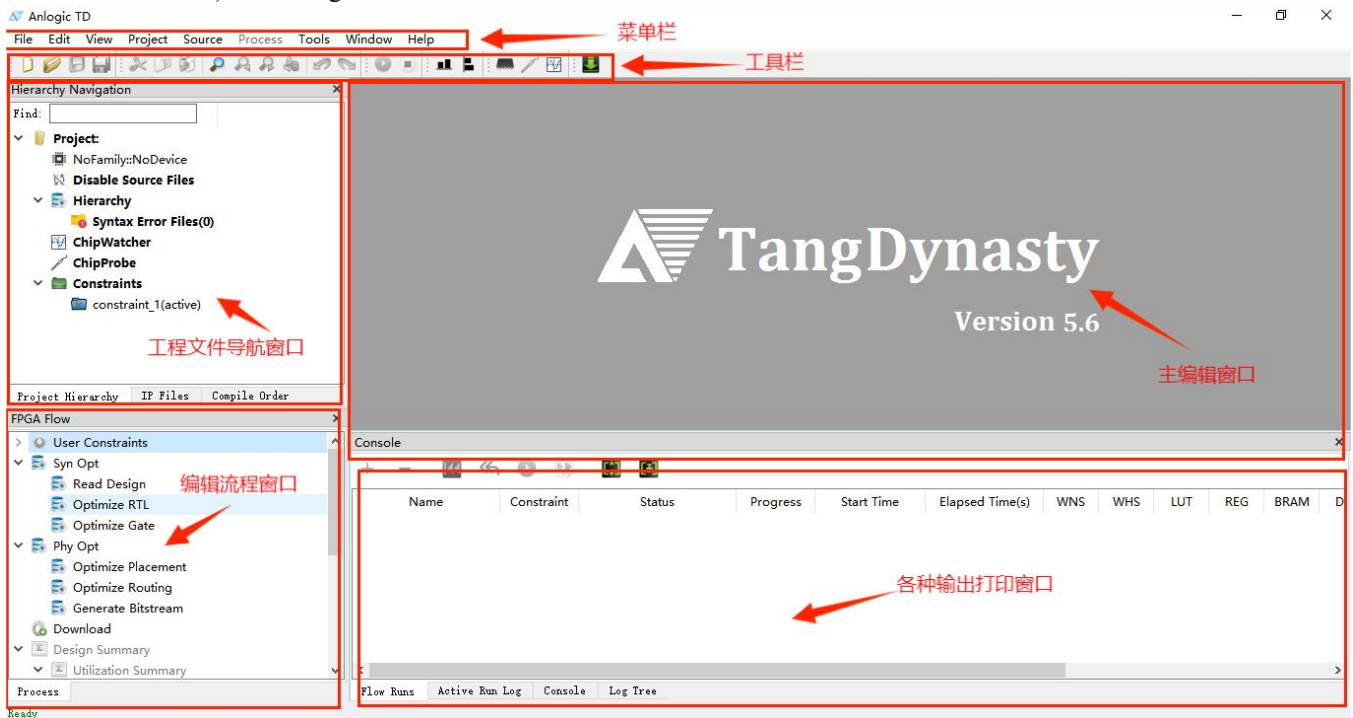


图 1-0

在主页面上选择【New Project】在弹出的窗口中点击【OK】，新建工程向导说明页面如图1-1 所示。

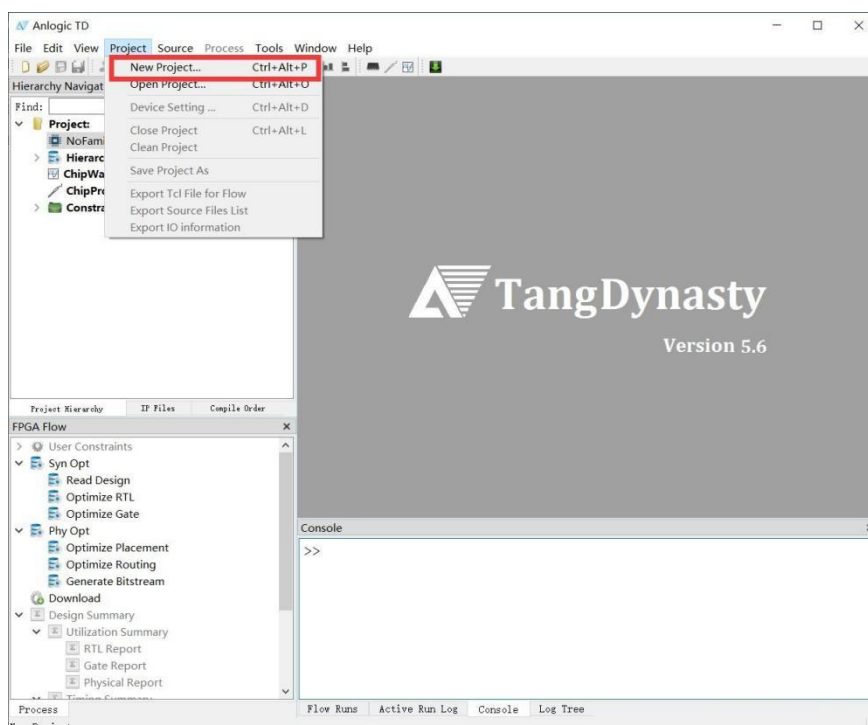


图 1-1

确定工程名。如图 1-2 所示，在打开的对话框中分别选择工程路径，设置工程名字。这里我的工程路径是 C:/ANLU/cnt1s，这里需要注意的是文件夹必须先自行建好，否则软件会报错“路径不存在”，另外 TD 的路径中不能出现中文名。接下来的工程名和顶层文件名取 cnt1s，然后点击“OK”。

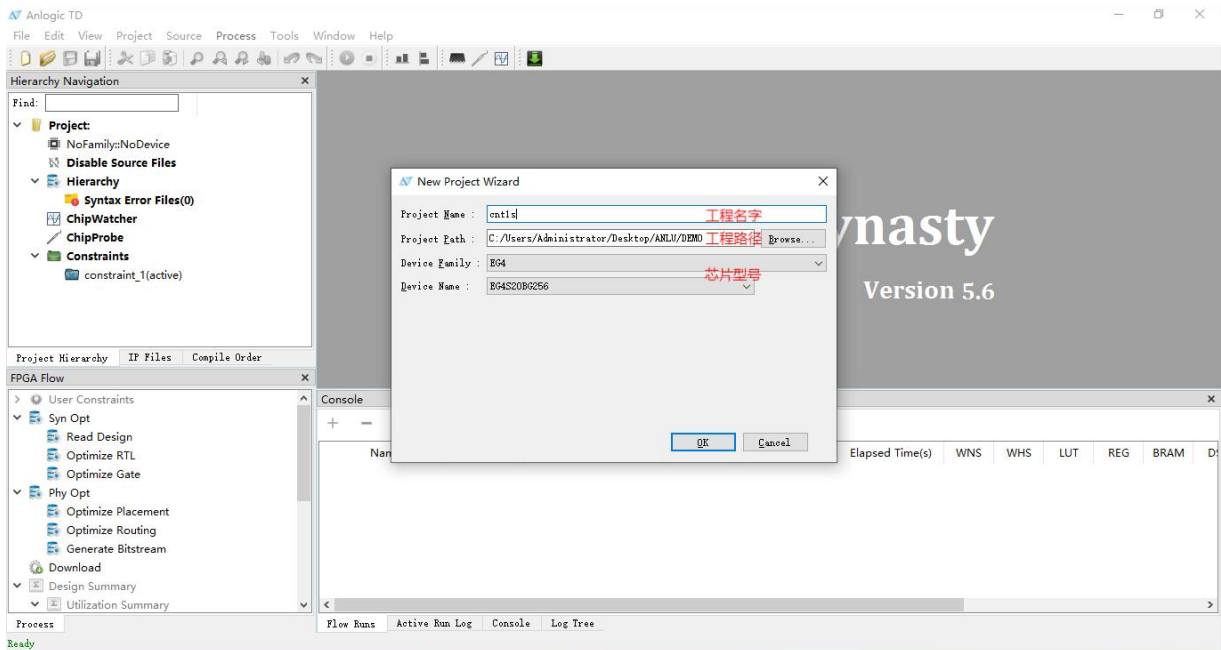


图 1-2

1.2 输入设计

添加已有的设计文件或新建设计文件。如图 1-3 所示，在这里我们选择新建设计文件，如添加现有的设计文件（事先将要使用的.v 文件拷贝到工程文件夹下）添加已有的设计文件后，TD 软件会自动识别顶层文件并将其设置为“Top”。

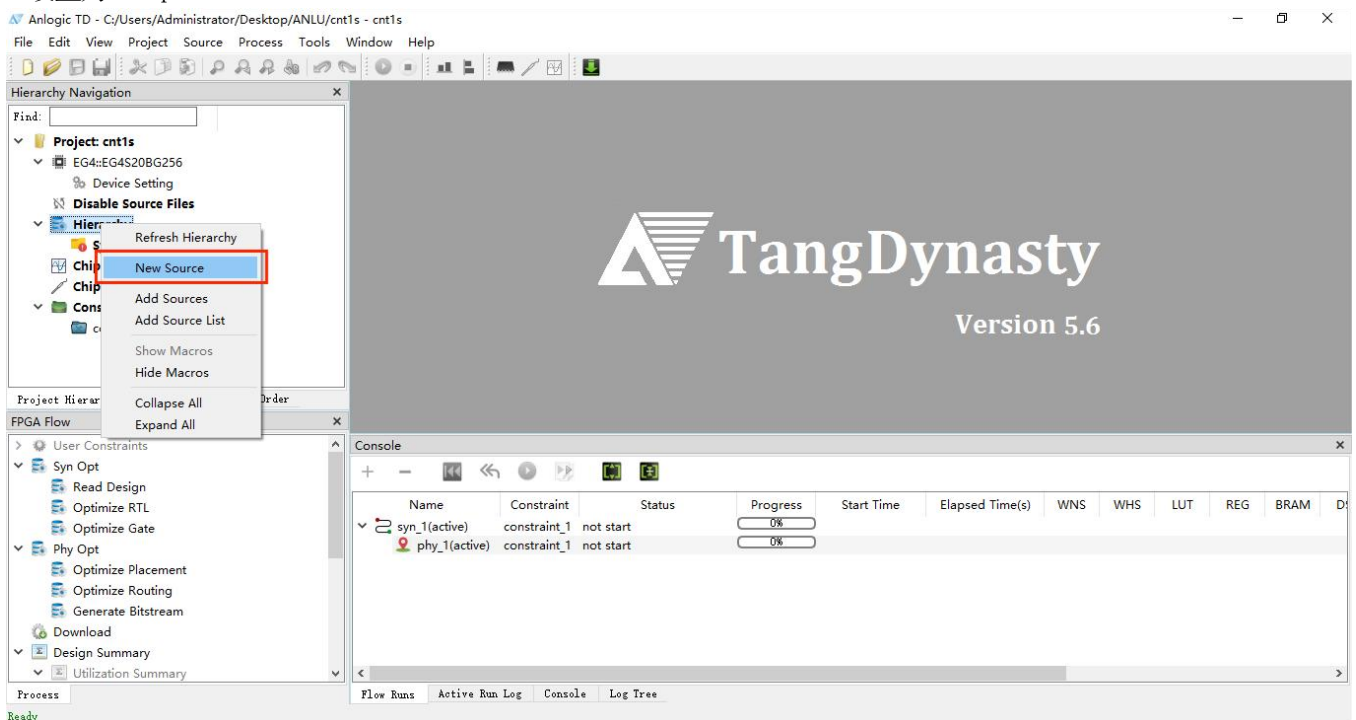


图 1-3

在图 1-3 上面选项区域中，右击【Hierarchy】→【New Source】。进入图 1-4 所示页面。

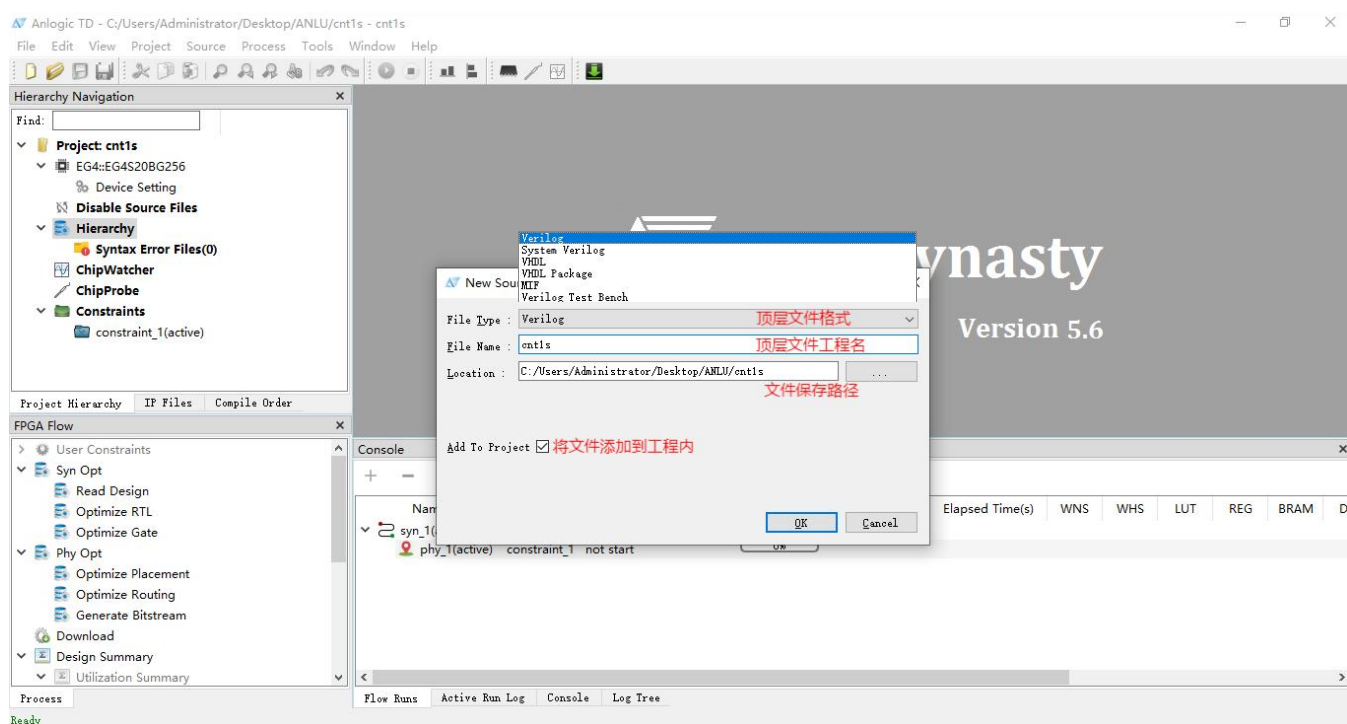


图 1-4

下面我们就来创建工程顶层文件，如图 1-4，在这里可以选择各种需要的设计文件格式。可以作为工程顶层设计文件的格式，选择“Verilog”或“VHDL”并勾选【Add to project】（将文件添加到工程内）单击 OK 按钮完成文件创建。（小提示，创建工程顶层文件我们还可以通过工具栏来创建，具体使用方法就不给出图片，读者可以用鼠标放置在工具栏的图标上会显示出其图标所标示的功能，工具栏使用方法和 word 功能相同，我们还可以通过快捷键【Ctrl】+【N】来创建工程顶层文件。

输入工程名 cnt1s，工程目录下会自动创建同名的工程文件，进入主编辑窗口，如图 1-5 所示

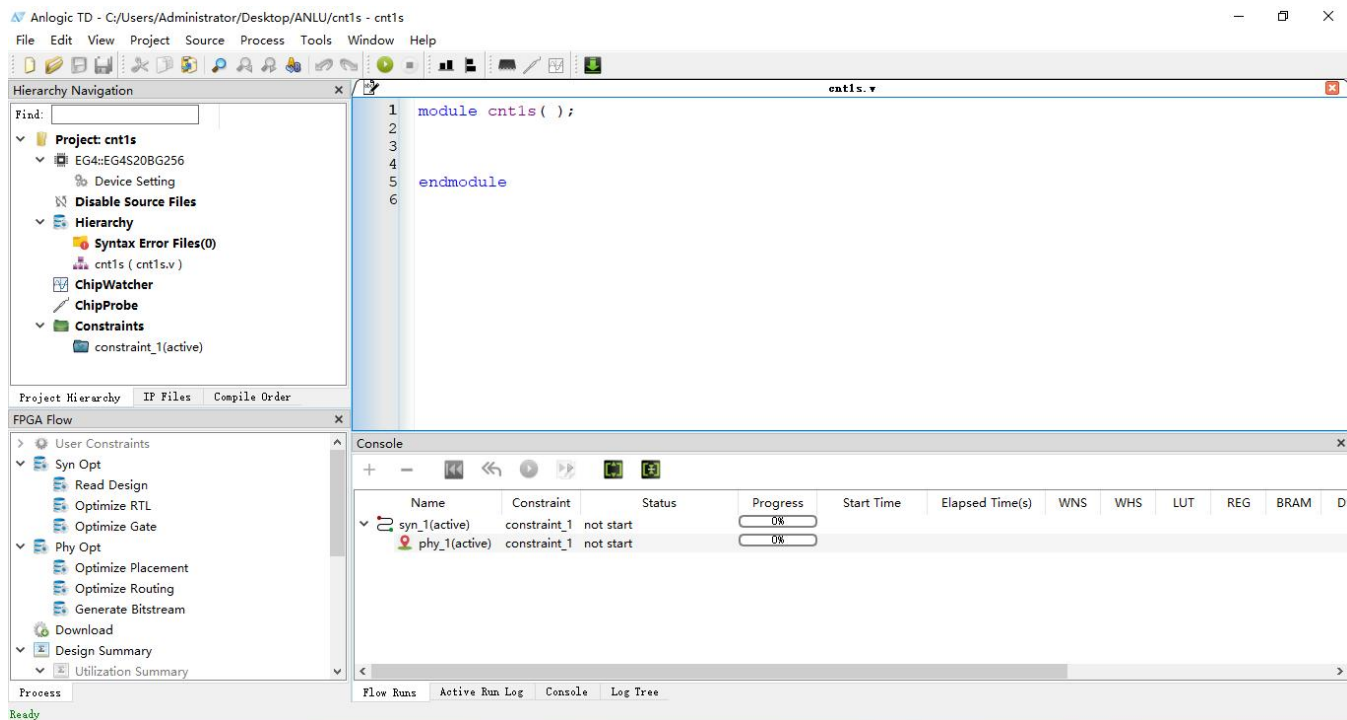


图 1-5

在主编辑窗口中，出现了一个新建的空白可编辑文件，接着在该文件中输入实现实验功能的 Verilog 代码，这个代码的功能为让核心板上的四个 LED 灯以 1s 的间隔闪烁，代码如下：

```

1 module cnt1s
2 (
3 input CLOCK,
4 input rst_n,
5 output reg [3 : 0] LED );
6 localparam T1S = 27'd49_999_999; //时钟为 50M, 故分频次数设为 50M-1
7 reg [26 : 0] cnt_value;
8 9 always @(posedge CLOCK or negedge rst_n)begin
9 if(!rst_n)
10 cnt_value <= 0;
11 else if(cnt_value == T1S)
12 cnt_value <= 0;
13 else
14 cnt_value <= cnt_value + 1'b1;
15 end
16 always @(posedge CLOCK or negedge rst_n)begin
17 if(!rst_n)
18 LED <= 4'Hf ;
19 else if(cnt_value == T1S) //两次 (cnt_value == T1S) 之间刚刚隔了 1S
20 LED <= ~LED;
21 end
22 endmodule
23

```

这里我们可以将上述源代码直接复制到主编辑窗口源代码，按快捷键【Ctrl】+【S】或选择【File】→【Save】，保存代码，文件名需要和代码中所命名的“cnt1s”名称一致，通常也都采用默认设置进行保存即可。如图 1-6 所示。

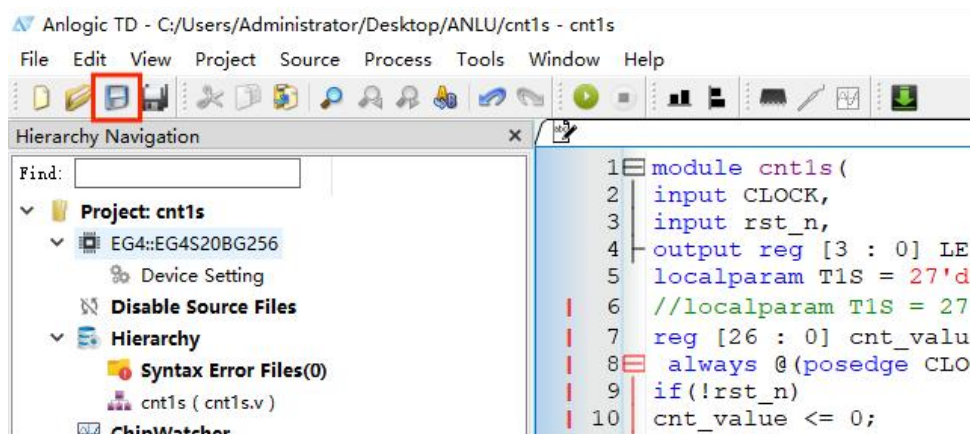


图 1-6

1.3 编译工程

至此，工程创建和设计输入工作已经完成。为了验证代码是否正确，我们可以双击【FPGA Flow】下的【Syn Opt】→【Read Design】，整个工程进行一次全编译，来验证基本语法是否正确。编译如果没有错误，通常【Syn Opt】左边会显示对勾，表示编译通过，如图 1-7 所示。

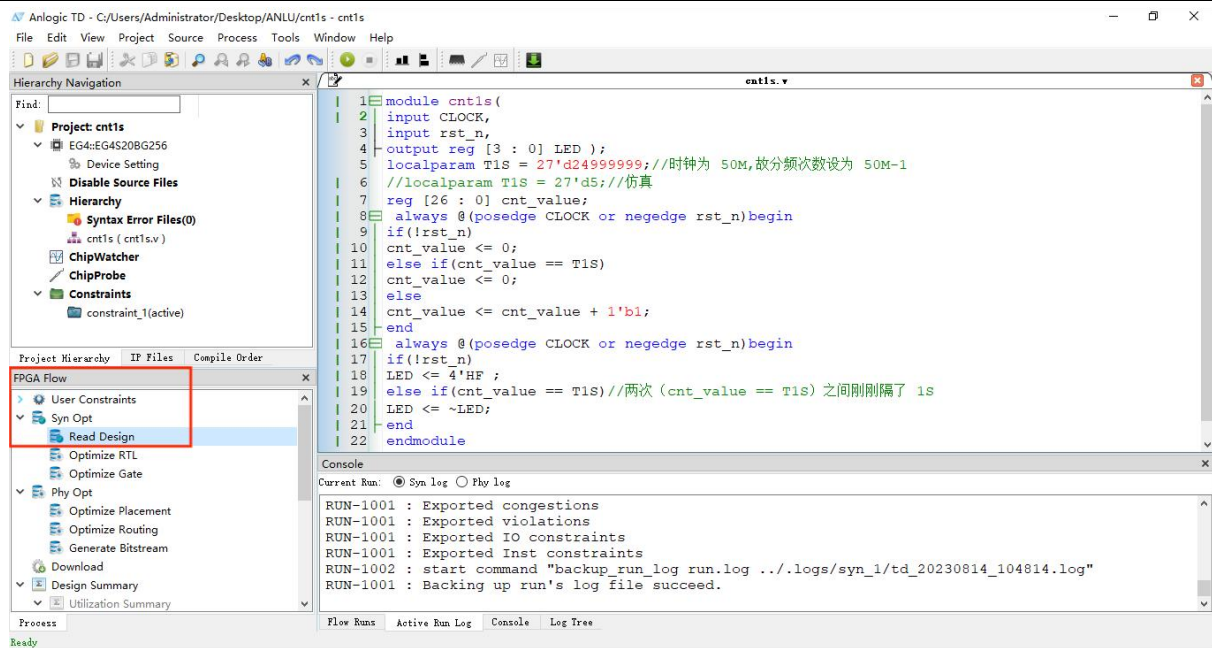


图 1-7

最后，我们可以查看打印窗口的“Console”的打印日志，如图 1-8 所示。包括“Critical”、“Warning”和“Error”。“Error”是不得不关注的，“Error”意味着代码有语法错误，后续的编译将无法继续；而“Warning”则不一定是致命的，很多时候“Warning”中暗藏玄机，很多潜在的问题都可以从这些条目中寻找蛛丝马迹。如果设计者确认这些“Warning”符合设计要求，或者说对我们的设计没有影响，那么我们也可以忽略它。



图 1-8

1.4 仿真工程

为了方便组织整篇教程，我们将仿真单独放在 § 2 ModelSim 软件中讲述。

1.5 分配管脚

编译（和仿真）通过以后，接下来我们就需要对工程中输入、输出端口进行管脚分配。管脚分配有两种方法：第一种就是手动分配，我们需要对照管脚分配表将管脚输入到对应的端口中，这种方法适用于一些少量的端口分配。第二种就是使用物理约束文件进行管脚分配，使用这种方法我们需要制作相应的脚本文件，这种方法适用于大量的端口分配。下面我们对这两种方法分别进行介绍。

首先我们介绍的是手动分配方法：我们可以使用菜单栏里【Tools】→【IO Constraint】对工程的 I/O 引脚进行分配，如图 1-9。

我们可以看到出现了三个端口分别是我们的时钟，复位和LED，如果你打开管脚分配页面，页面中并没有出现顶层文件的端口信号，那么我们建议大家先对整个工程执行一次全编译。在这个例程中，我们需要锁定 CLOCK 和 4 个 LED 管脚和复位脚 rst_n，利用我们手头上 KX-EG420 板，如图 1-10，此板核心板中输入输出脚。

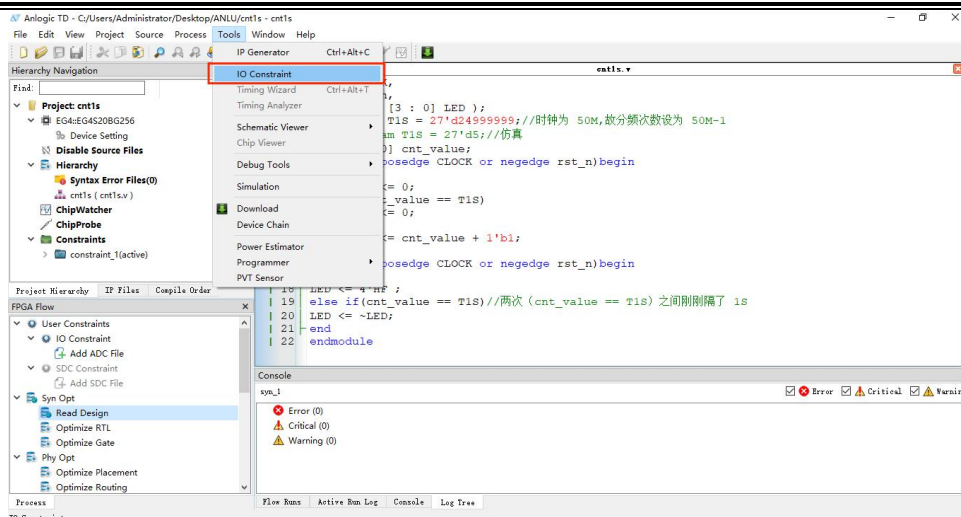


图 1-9

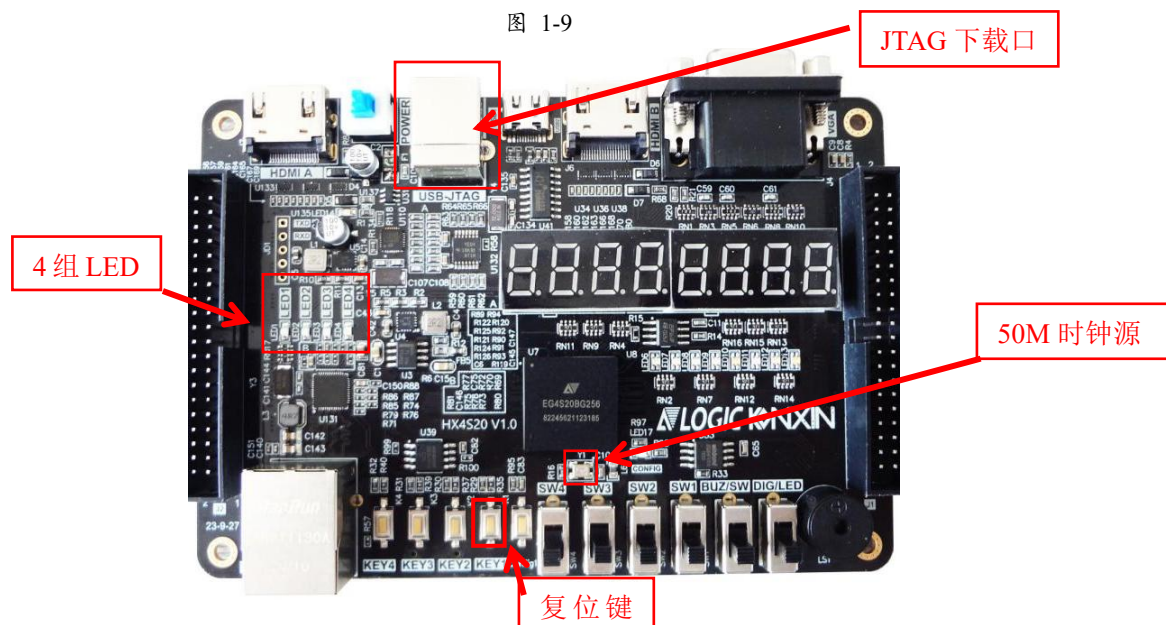


图 1-10 HX4S20

核心板上 CLK 时钟和 4 个 LED 他们的管脚请参阅（开发板手册“二章”介绍）。经查核心板 CLOCK 和 4 个 LED、复位引脚如下：

引脚名称	FPGA 引脚
CLK	R7
复位按键名称	
rst_n (K1)	A2
LED名称	
LED0	A4
LED1	A3
LED2	C10
LED3	B12

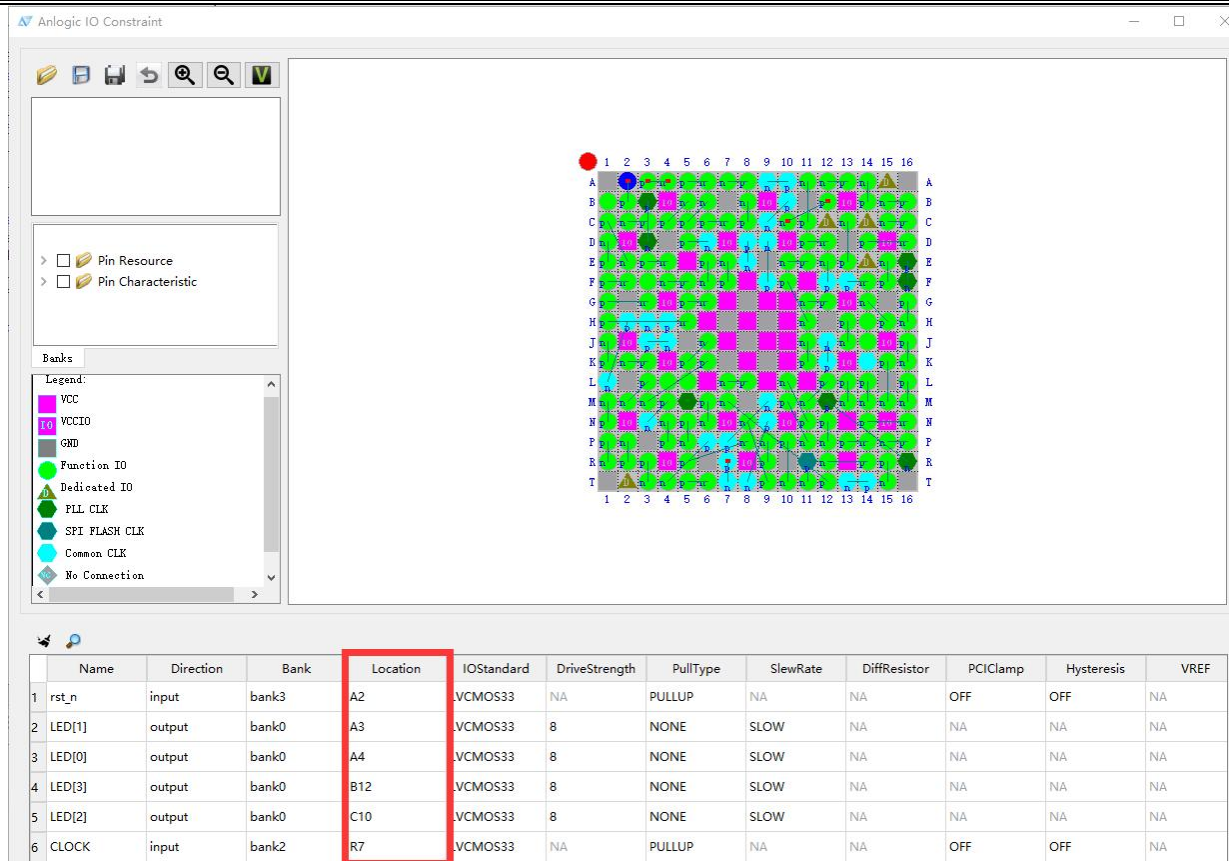


图 1-11

接下来我们就可以参考管脚分配表，将它们对应的引脚号输入到对应信号名的“Location”中，如图 1-11 所示。（这里再啰嗦几句说一下如何输入：首先我们用鼠标点击 CLOCK 信号名 Location 下面的空白位置，我们可以输入 PIN_R7，也可以直接输入 R7 接下来我们点击回车。这样我们就完成了管脚分配，这里需要根据原理图的硬件连接关系分配引脚，点击保存 Anlogic 会将图形化设置保存为 xxx.adc 文件(需自定义文件名)，并自动将该 adc 文件（Anlogic Design Constrains）添加到工程中。如图 1-12 所示。

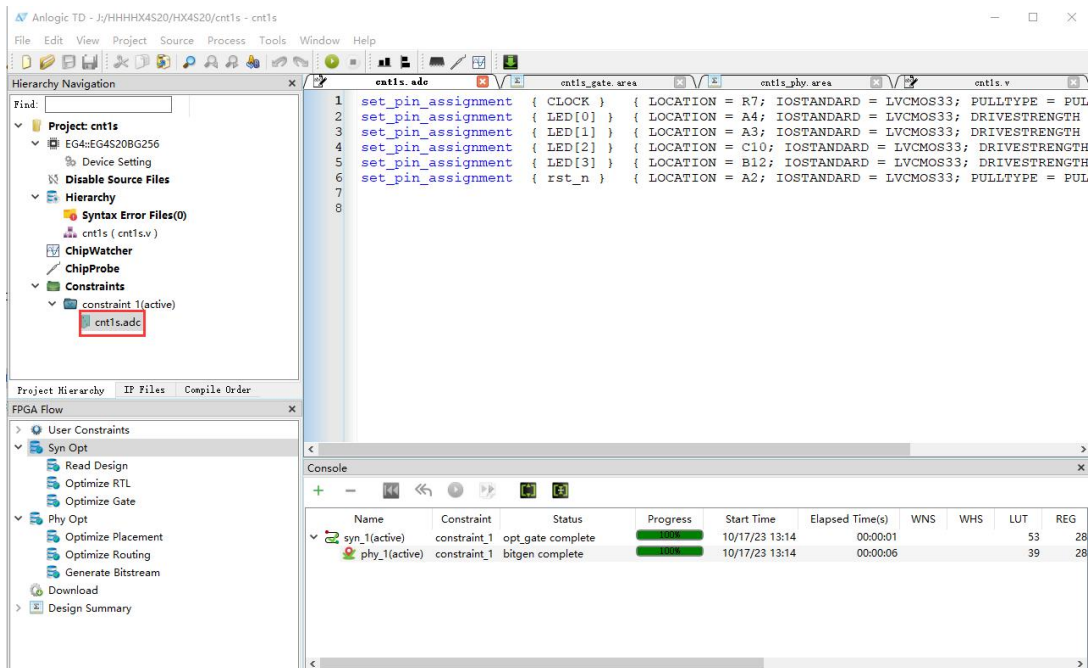


图 -12

对于一些比较少的管脚可以进行手动分配，但是碰到很多或者要经常分配管脚，这种方法显然是比较繁琐。后续

设计,可以使用另一种方法,直接修改 adc 文件,并手动将 adc 文件添加到工程下。例如图 1-12 是刚才生成的 adc 文件,这里有个地方需要大家注意下, **Adc 文件无法自行创建,通过 IO 分配界面锁定引脚后会生成 Adc 文件**,保存后会自动添加到工程当中,可以根据新的工程手动修改网络名和 IO 位置,另存后给其他工程使用。Anlogic 的约束文件是以“.Adc”为文件后缀的文本文件,其中存储的是一条条的约束命令。我们在.Adc 文件内输入如下内容

```
set_pin_assignment { CLOCK } { LOCATION = R7; IOSTANDARD = LVCMOS33; PULLTYPE = PULLUP; }
set_pin_assignment { LED[0] } { LOCATION = A4; IOSTANDARD = LVCMOS33; DRIVESTRENGTH = 8; PULLTYPE = NONE; }
set_pin_assignment { LED[1] } { LOCATION = A3; IOSTANDARD = LVCMOS33; DRIVESTRENGTH = 8; PULLTYPE = NONE; }
set_pin_assignment { LED[2] } { LOCATION = C10; IOSTANDARD = LVCMOS33; DRIVESTRENGTH = 8; PULLTYPE = NONE; }
set_pin_assignment { LED[3] } { LOCATION = B12; IOSTANDARD = LVCMOS33; DRIVESTRENGTH = 8; PULLTYPE = NONE; }
set_pin_assignment { rst_n } { LOCATION = A2; IOSTANDARD = LVCMOS33; PULLTYPE = PULLUP; }
```

在工程界面选择【Constraints】→【Add ADC File(s)】添加已有的约束文件,如图 1-13

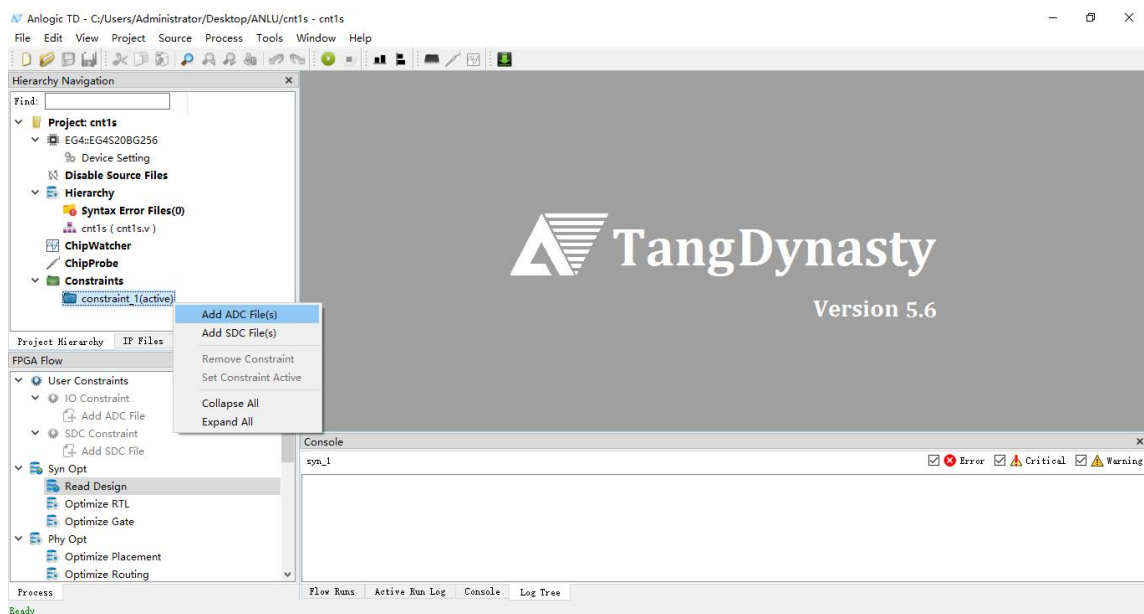


图 1-13

选中.adc 文件,点击打开,如图 1-14 所示,然后在进行一次编译,就完成了约束的输入,可以打开 IO 引脚分配界面,查看所编辑的引脚是否正确。

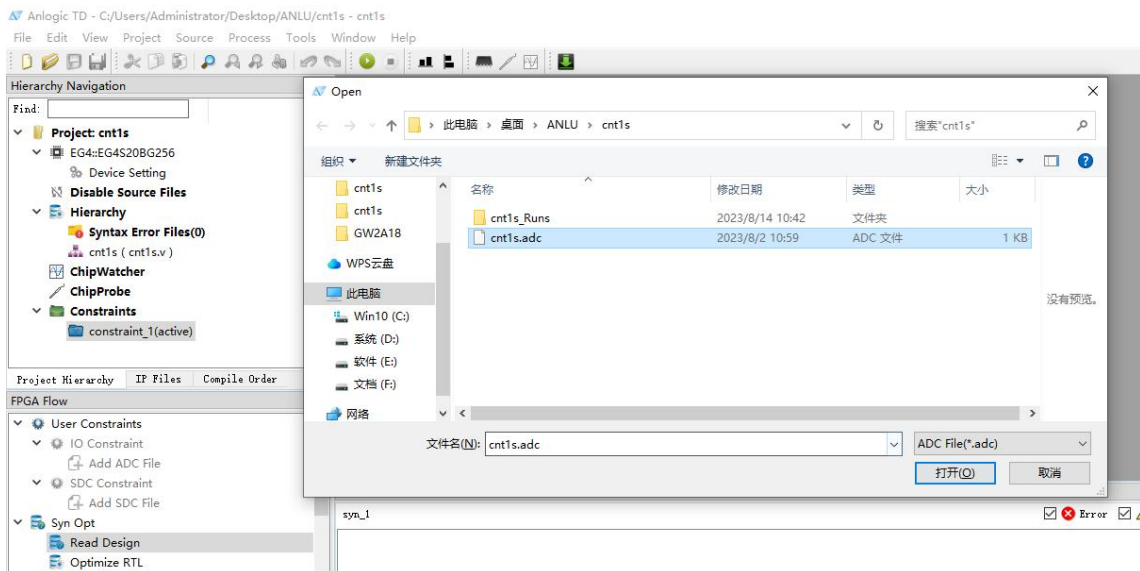


图 1-14

1.6 编译布线

要下载设计，首先要生成用于下载到器件中的比特流文件。如图 1-15 所示，点击【Run】。

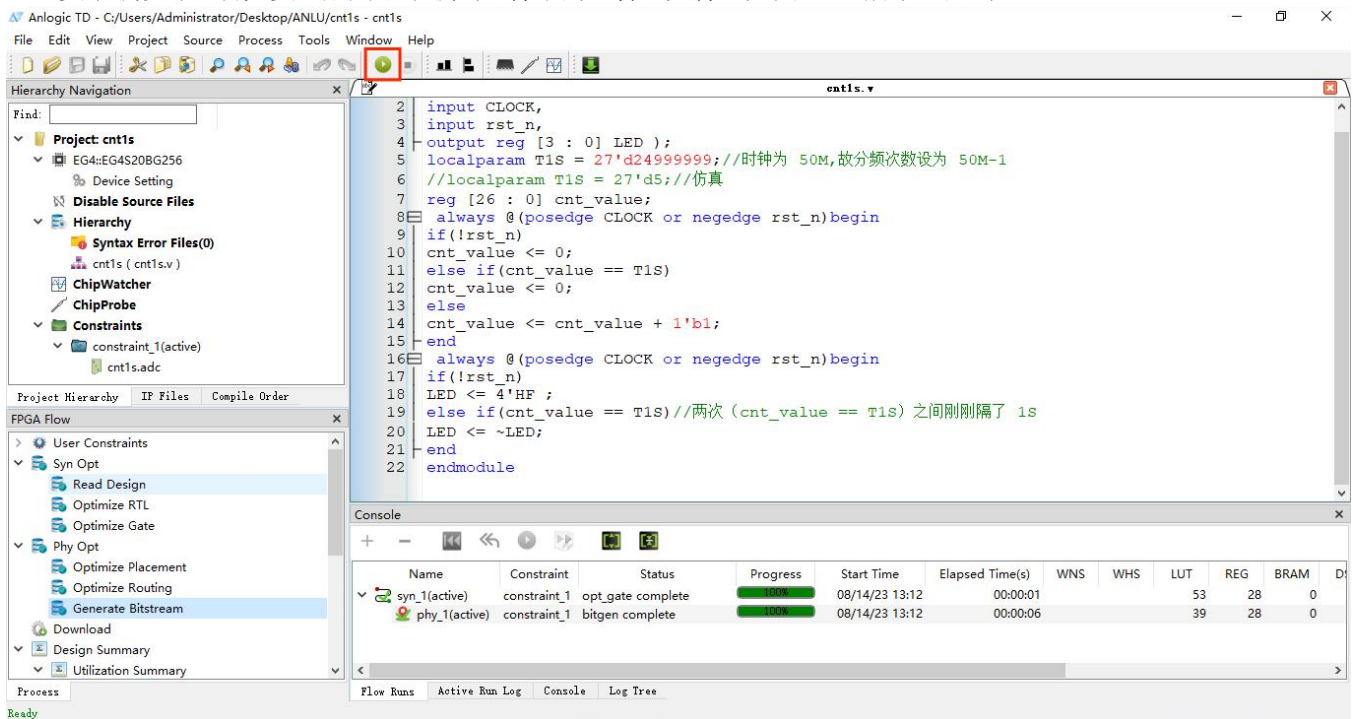


图 1-15

1.7 下载程序

编译完成后，我们就可以给核心板下载程序，来验证我们的程序能否正常运行。首先要连接好开发板，打开红色按钮电源开关，插好下载线。如图：1-10 核心板的 USB 接口。

我们在工具栏上找到【Download】点击【Download】如图 1-16 所示，点击【Download】进入下载界面

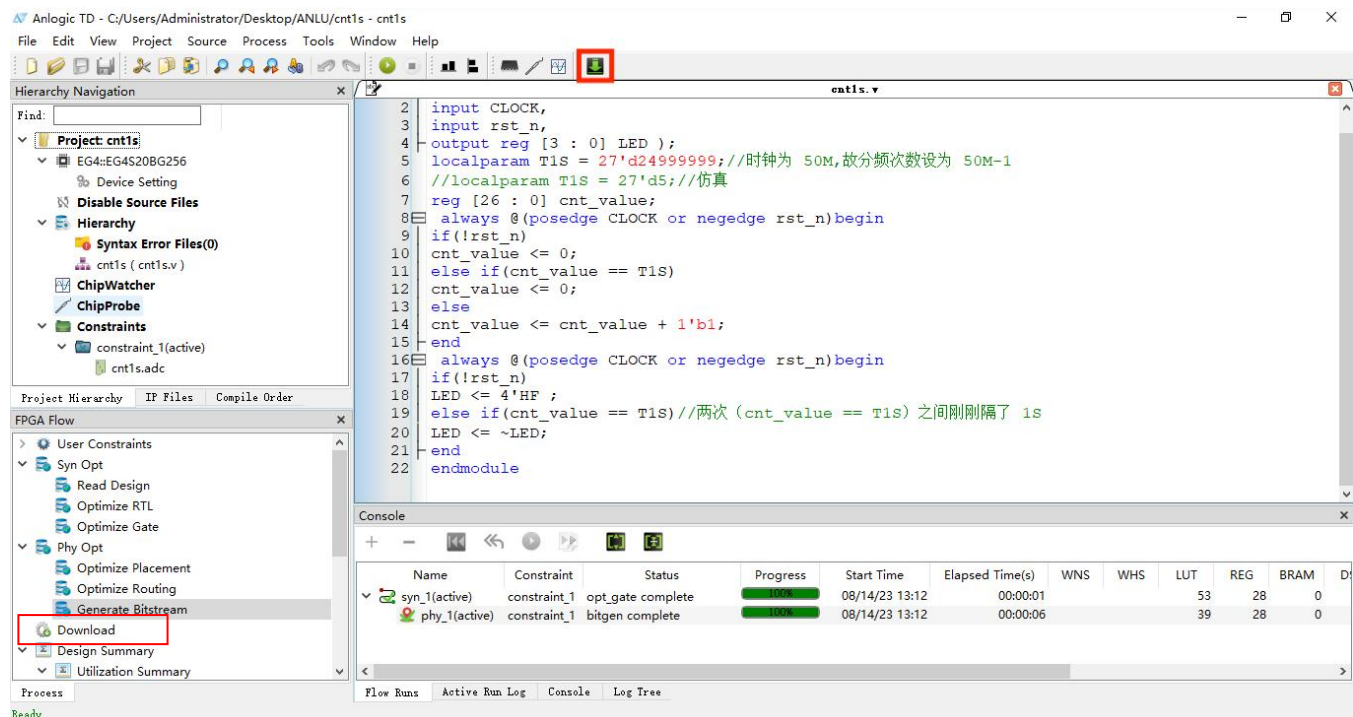


图 1-16

首次使用该开发板时，需要安装 USB-JTAG 驱动程序

右键桌面图标“计算机”，点击“属性”，打开“设备管理器”：在通用串行总线设备下会出现“USB-JTAG-Cabl
如图 1-17 所示

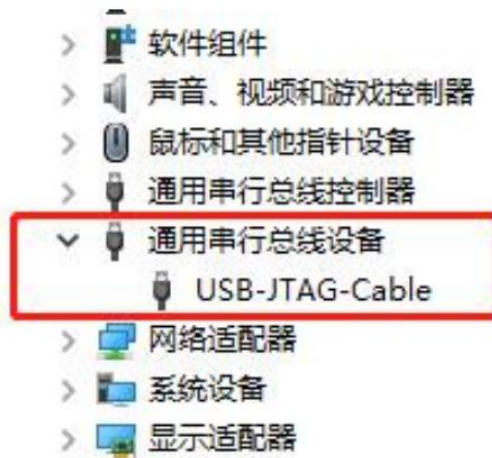


图 1-17

右键选择“USB-JTAG-Cable”，点击“更新驱动程序”：如图 1-18 所示，选择“浏览计算机以查找驱动程序软件”；如图 1-19 所示，点击“浏览”，定位到 TD 文件夹下“driver”里面的 win8_10_64 文件夹，并勾选“包括子文件夹”，点击“下一步”；如图 1-20 所示。若出现下图 1-21 中对话框则表明驱动安装成功。



图 1-18



图 1-19

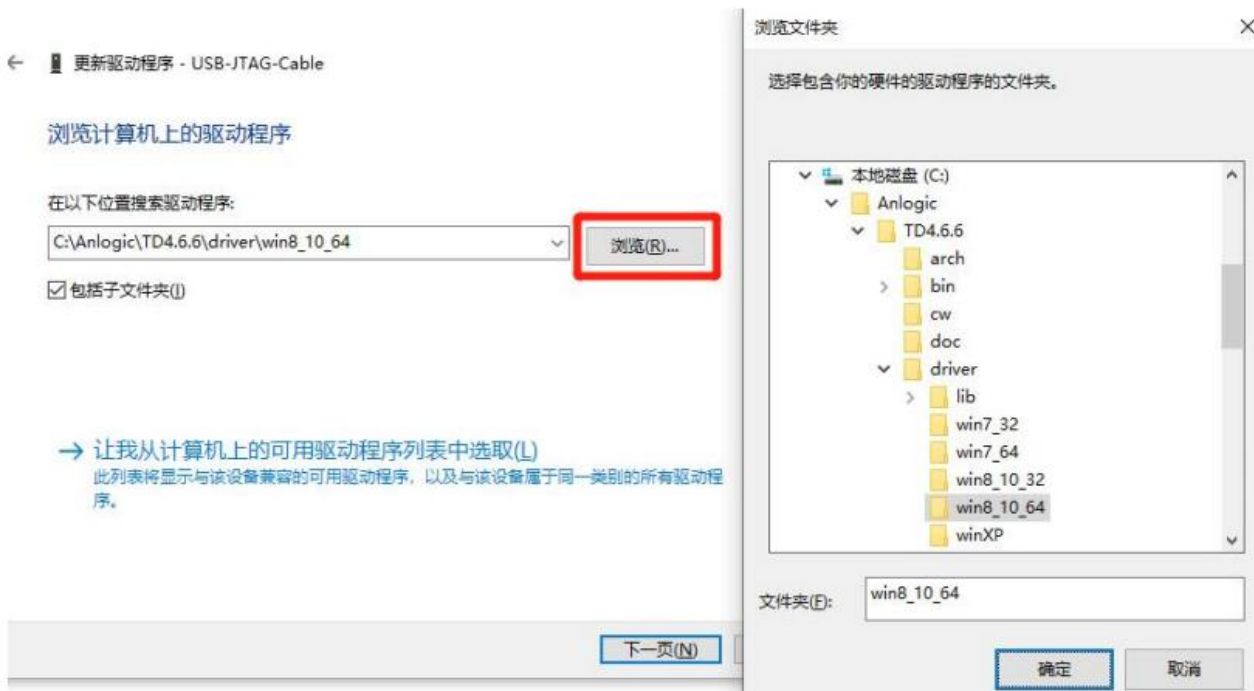


图 1-20

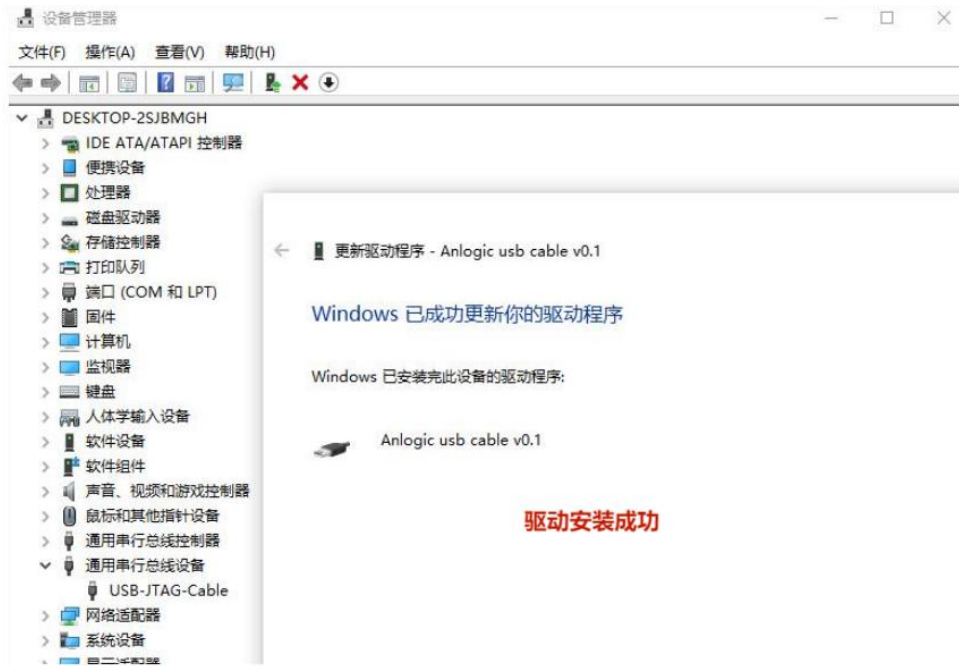


图 1-21

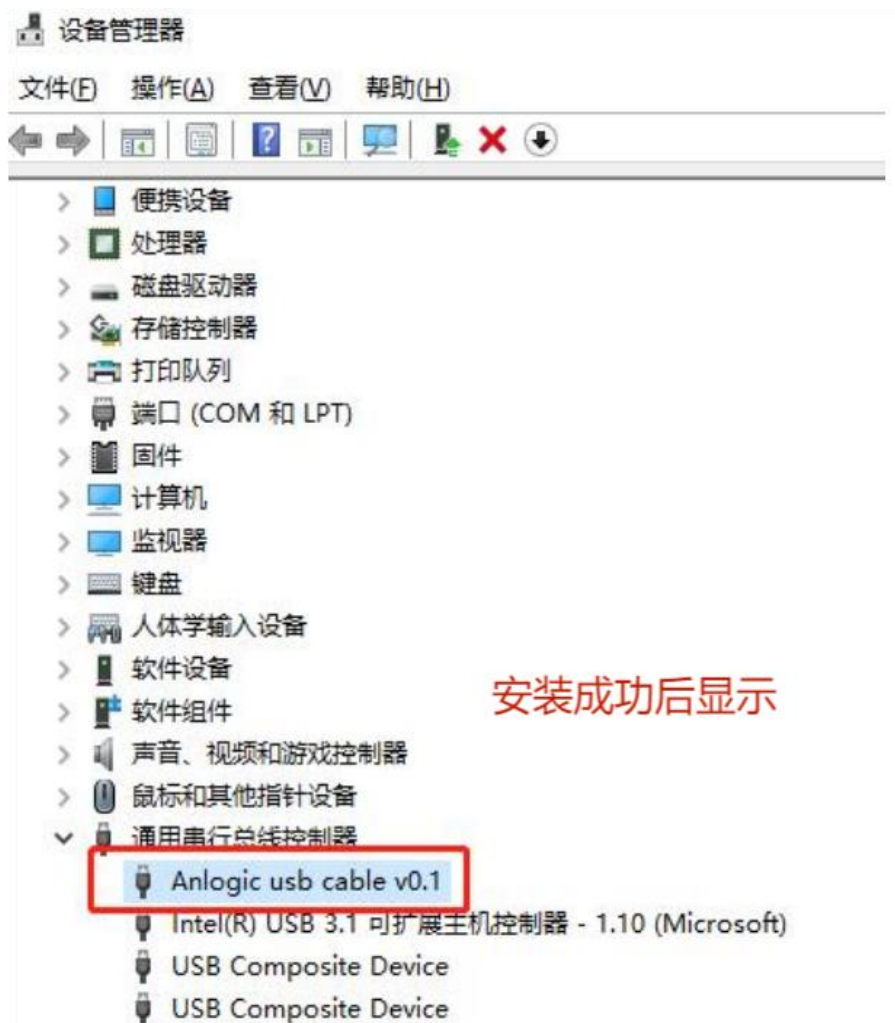


图 1-22

注意，有的电脑装好后 Anlogic USB Cable 上会有一个黄色叹号，导致 TD

软件不识别硬件，此时需要首先在 Bios 里禁用“Secure Boot”，然后进入 windows “禁用 WIN10 的驱动程序数字签名”，这样每次 Windows 重启都不再检测驱动程序的数字签名。

在弹出的下载页面查看是否自动识别到芯片，选择下载速度，如图 1-23

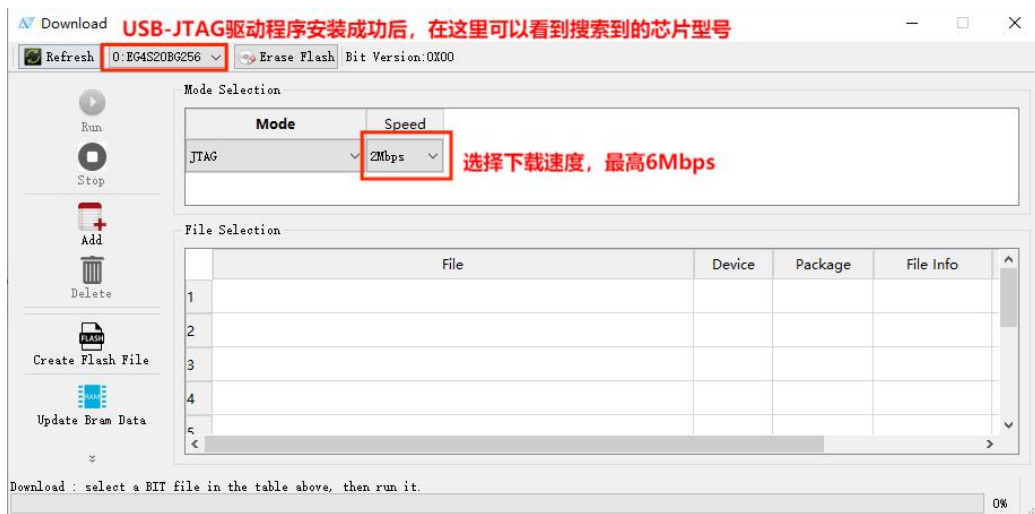


图 1-23

添加.bit 文件，注意文件路径，如图 1-24 所示

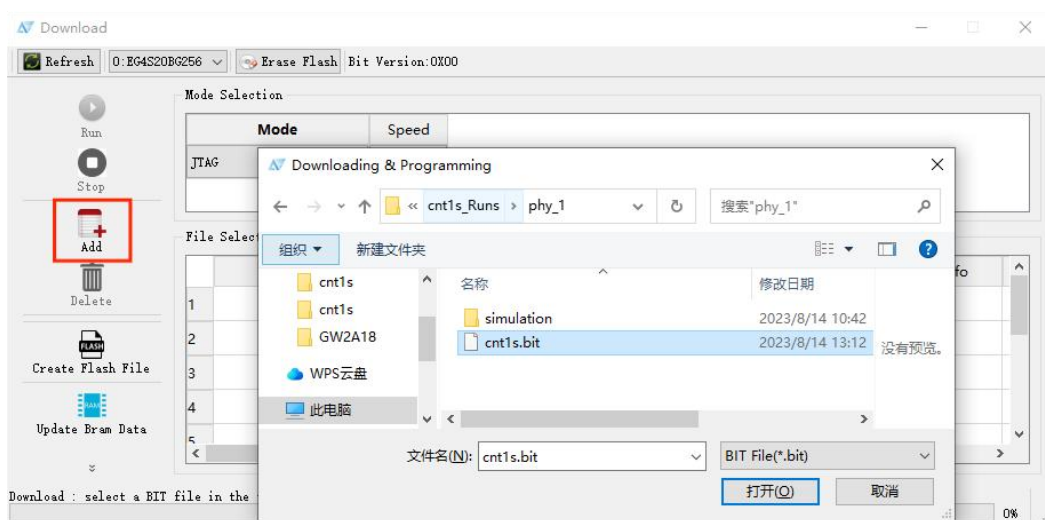


图 1-24

选中对应 bit 流，点击 run 下载程序，如图 1-25 所示

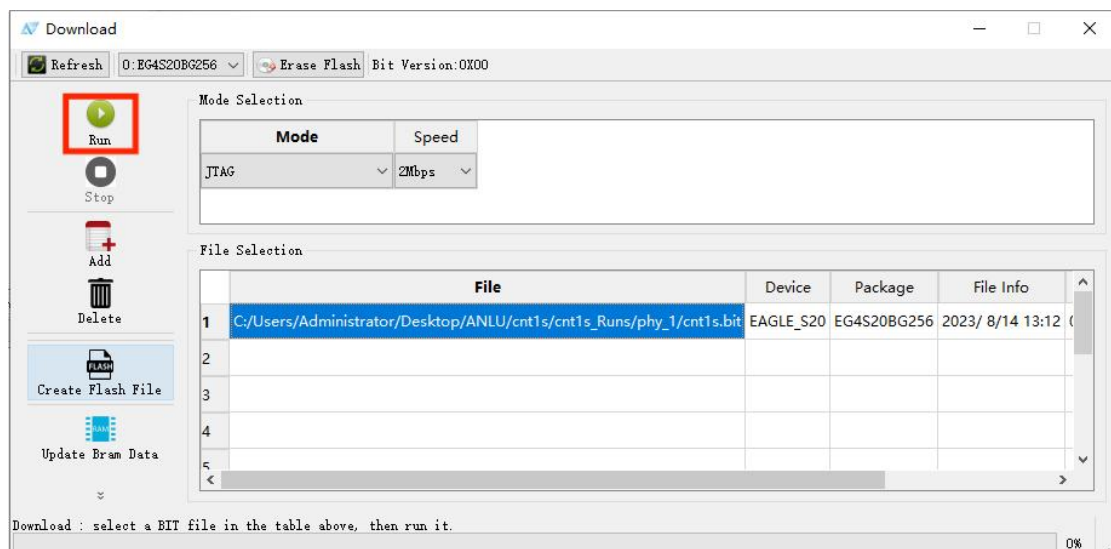


图 1-25

为了直接对 FPGA 进行下载（配置），在设备配置窗口的编程模式 SRAM Mode 中选择 JTAG（默认），注意要仔细核对下载文件路径与文件名，确定就是当前工程生成的编程文件（注意此文件所在路径的文件夹是 cnt1s\cnt1s_Runs\phy_1）。如果此文件没有出现，检查所在文件夹或重新编译。当下载进度显示出 100%时，表示编程成功。图 1-26

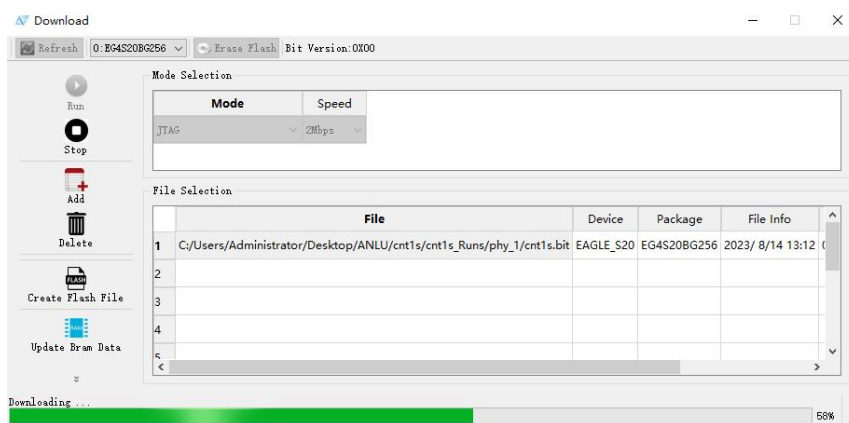


图 1-26

下载完成之后，我们可以看到开发板上的 LED1 指示灯欢快地亮了起来。

1.8 固化程序

可能已经有同学发现下载 .bit 文件到 FPGA 后，核心板重新上电后配置程序已经丢失，还需要 JTAG 下载。这岂不麻烦！好吧，这一节我们来介绍如何把配置程序固化到开发板上的 FLASH 中，这样不用担心掉电后程序丢失了。在我们的开发板上有一个 8Pin 的 64Mbit 的 FLASH，用于存储配置程序。下面为大家介绍 FLASH 程序的固化的流程。

这个下载程序是下载的 .bit 文件，开发板断电在上电程序将会丢失。如果我们想要下载了程序之后开发板断电在上电程序仍然存在，只需要在设备配置界面 Mode 选择【PROGRAM FLASH】模式，其余选择默认即可，然后点击【Run】如图 1-26 所示。

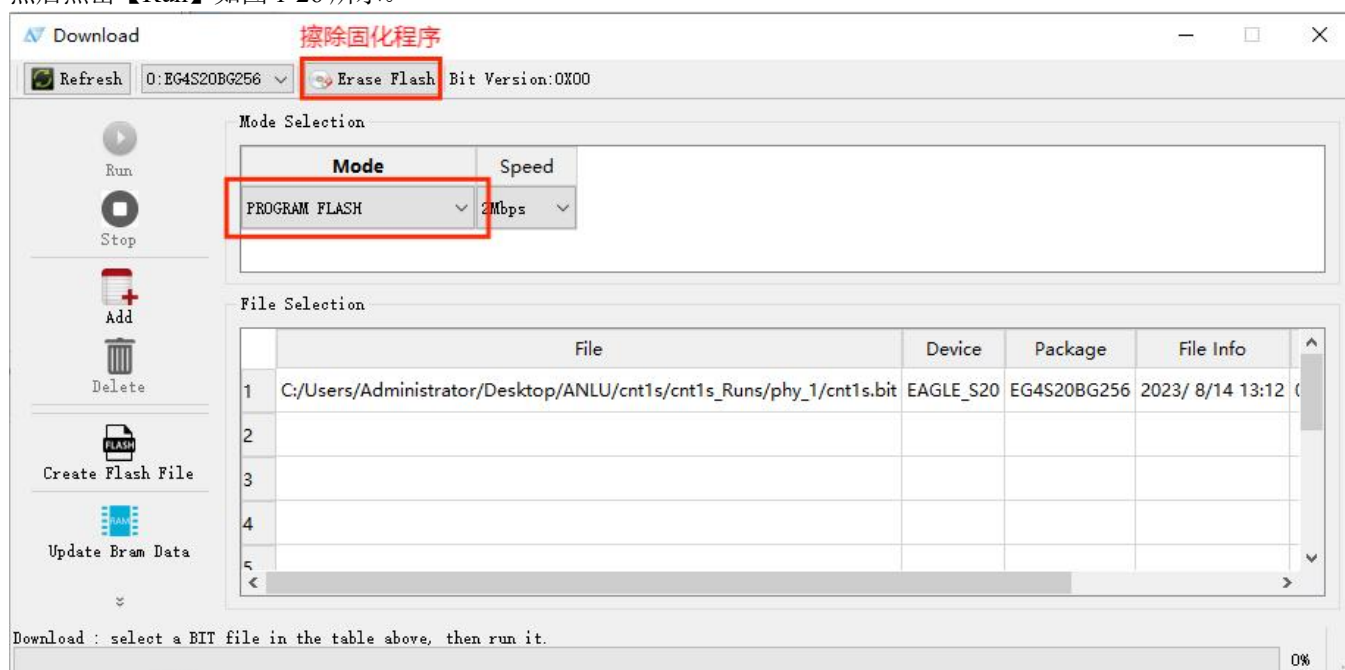


图 1-26

意外断电导致片外 Flash 中文件异常，导致 FPGA 不能正确加载。需要用 Erase Flash 模式擦除一遍片外 Flash 在进行下载。

第 2 章 ModelSim 工程仿真

在本文中你将能够学到：

- 1 在Anlogic 中调用 ModelSim 实现功能仿真的基本方法。

- 2 在这篇教程中，笔者将以一个简单的 LED 闪烁灯为例子，从工程实战的角度，带大家熟悉基于 Anlogic 和 ModelSim 最基本的使用方法。

要说明的是，虽然教程是基于 Anlogic 和 Modelsim SE-64 10.1c 编写的，但其内容也可以适用于较早的软件版本。在介绍 ModelSim 的使用流程之前，我们需要给大家说明一下仿真的两个概念，仿真一般分为前仿真与后仿真：

- (1) 前仿真也就是纯粹的功能仿真，主旨在于验证电路的功能是否符合设计要求，其特点是不考虑电路门延迟与线延迟。

- (2) 后仿真也就是时序仿真。是指电路已经映射到特定的工艺环境下，综合考虑电路的路径延迟与门延迟的影响，验证电路在一定时序条件下是否满足设计构想的过程。

说完了仿真的两个概念，接下来我们在来说一说 ModelSim 的使用，ModelSim 的使用主要分为两种情况，第一种就是通过 Anlogic 软件调用我们的 ModelSim 来进行仿真，这种情况也就是我们通常所说的自动仿真。第二种情况就是直接打开 ModelSim 进行仿真，不经过我们的 Anlogic 软件，这种情况也就是我们通常所说的手动仿真。

本篇教程中我们简单的介绍一下手动仿真，在 Anlogic TD 中是无法调用 ModelSim 软件来进行自动仿真，下面我们就以上章节中我们创建的 Anlogic TD 软件工程为例，进行仿真。

编写TestBench 仿真文件

在仿真前，我们需自行编写 TestBench 仿真文件，在 Modelsim 软件我们要做的就是在这个模板里添加我们需要的测试代码（也就是我们常说的激励）。下面我们给出修改完毕的代码。

```
1 `timescale 1 ns / 1 ns
2 module tb_cntls();
3
4 reg CLOCK;
5 reg rst_n;
6 wire LED;
7
8 cntls i1 (
9     .clk_50M(CLOCK),
10    .led_out(LED),
11    .rst_n(rst_n)
12 );
13
14 initial
15 begin
16     $display("Running testbench");
17
18     CLOCK = 0;
19     rst_n = 0;
```

```

20 #20 rst_n = 1;
22 end
24 always
25 begin
26 #10 CLOCK = ~CLOCK;
27 end
29 endmodule

```

2.1 配置仿真工程

打开 Modelsim 点击【File】→【New】→【Project】，新建一个工程，如图 2-1 所示

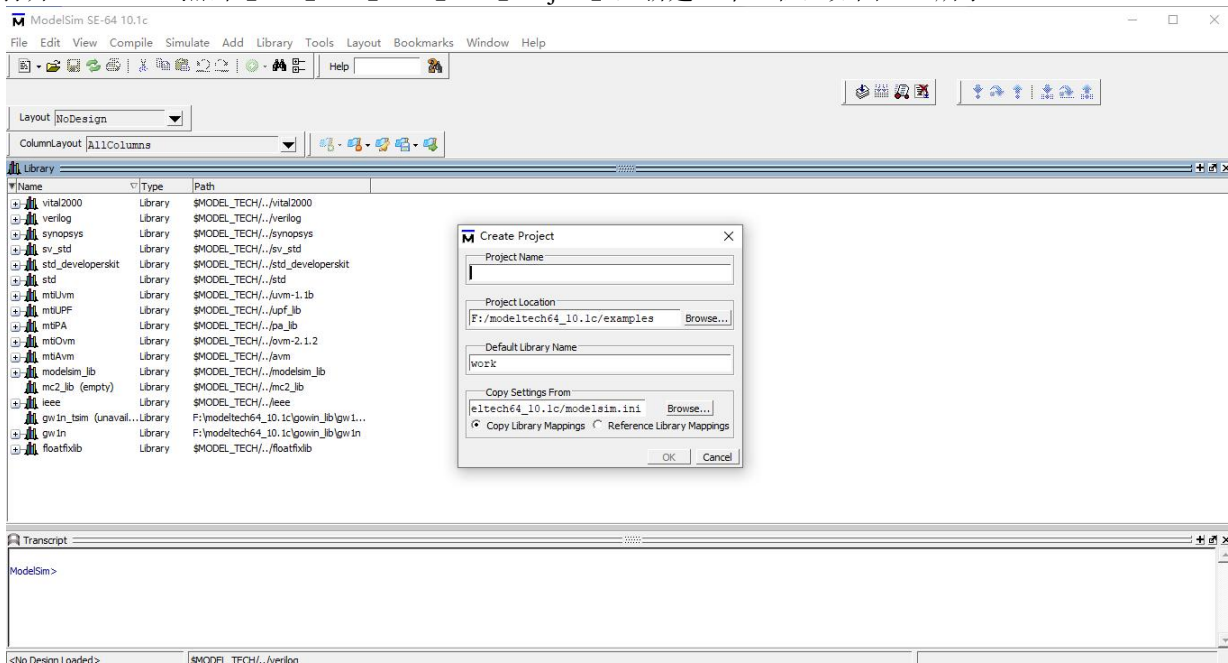


图 2-1

输入工程名 cnt1s，将 Modelsim 工程保存在 cnt1s 工程例子文件夹下面，点击【OK】，如图 2-2 所示

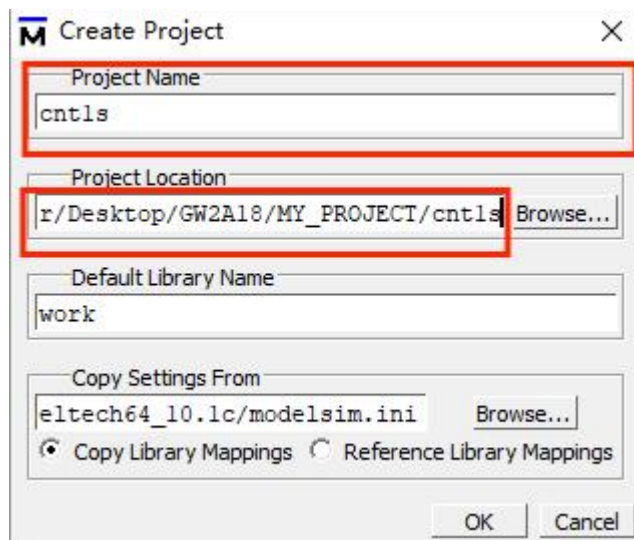


图 2-2

在弹出的界面，点击【Add Existing File】将工程源文件与 TestBench 仿真文件添加进工程，点击【OK】，如图 2-3、2-4 所示

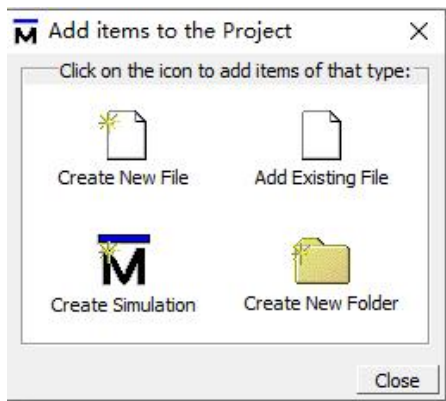


图 2-3

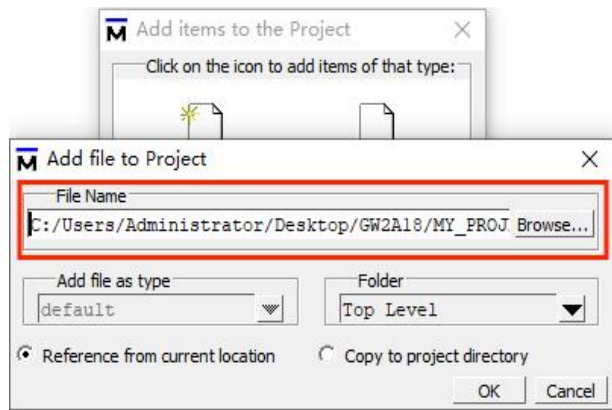


图 2-4

在工程界面，可以看到两个源文件，我们点击【Compile】→【Compile_All】来编译这两个文件，在编译通过后，通常问号会变成对勾，表示编译通过，如图 2-5、2-6 所示

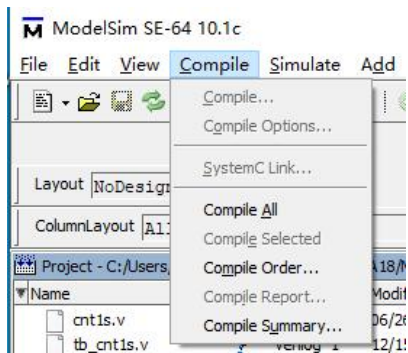


图 2-5

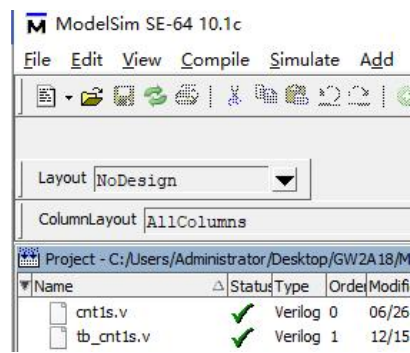


图 2-6

2.2 开始功能仿真

万事俱备只欠东风，经过上面这么多的操作步骤，我们终于完成了所有设置，接下来我们就可以进行仿真了，在开始仿真之前，我们这里有一点要需要注意，我们在 Anlogic 软件中实现的功能是 LED 闪烁，它的间隔时间是 1s，如果我们想要仿真这个功能，那么我们仿真软件运行时间最低就是 2s。这 2s 钟在我们看来是很短很短的，不过在仿真软件看来是很长很长的，我们的仿真软件单位可是 ns。为了便于我们仿真，这里我们需要稍微改动一下 Verilog 代码，我们需要将 `localparam T1S = 27'd49_999_999;` 修改为 `localparam T1S = 27'd5;`

修改完毕后，我们在 Modelsim 软件界面中的菜单栏中找到【Library】→【Work】→【tb_cnt1s】按钮并右击，选择【Simulate】，则会出现如图 2-7 所示界面。

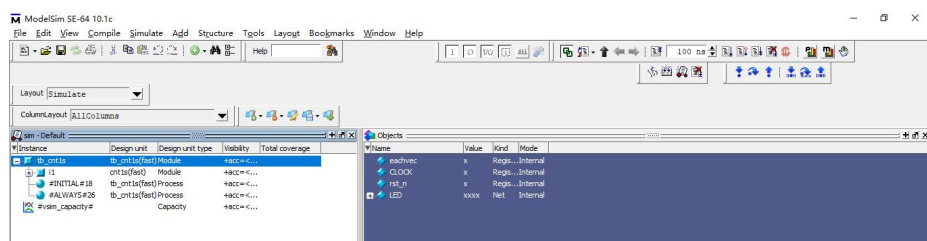


图 2-7

右击选择【tb_cntls】→【Add_To】→【Wave】→【All items in region】将信号添加进去，如图 2-8 所示

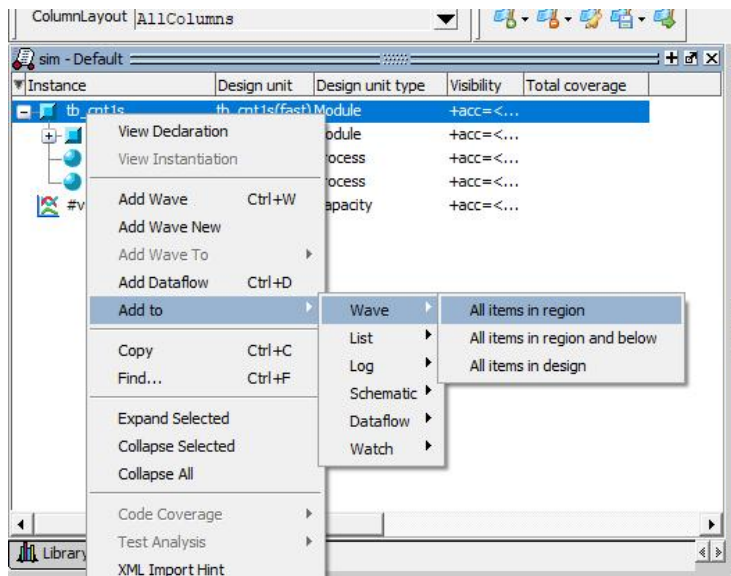


图 2-8

在弹出的界面下的副本窗口输入仿真时长“1000ns”然后按回车，如图 2-9 所示

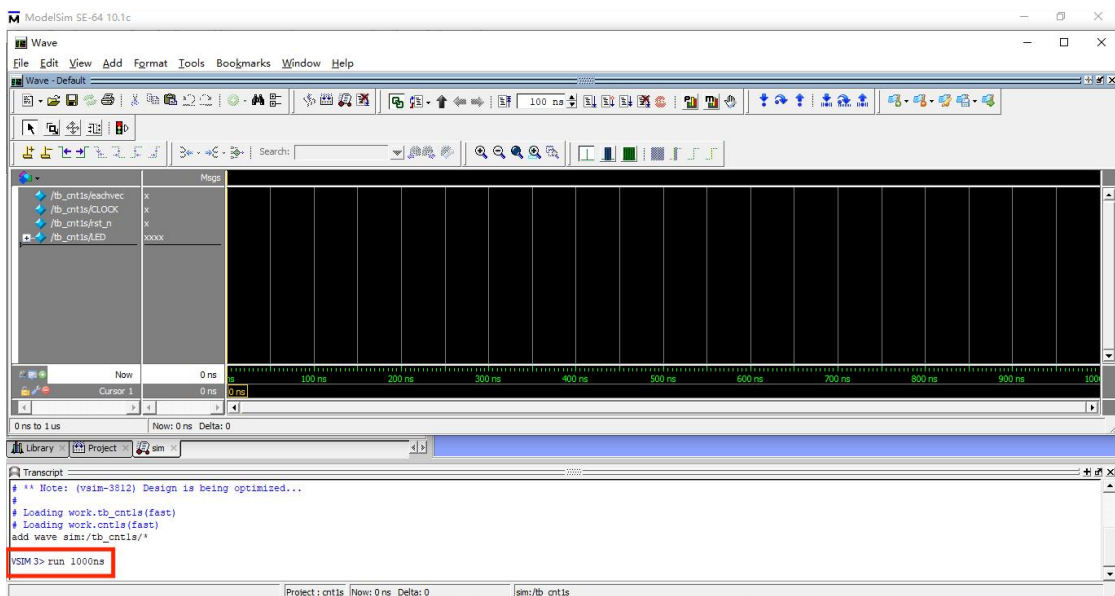


图 2-9

此时就可以看到 Modelsim 仿真出的波形，如图 2-10 所示

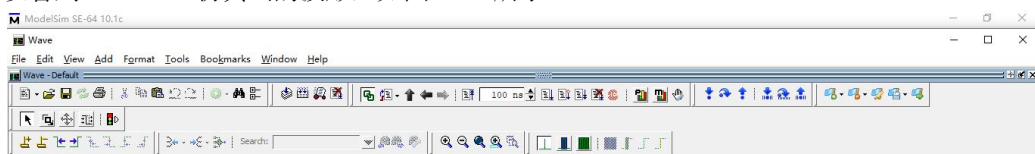


图 2-10

在该软件启动过程中，我们不需要任何操作，它会自动完成仿真，并给出我们所需要的波形，当波形图出现之后，我们就可以查看波形来判断设计功能是否正常了。为了方便大家更容易观察波形，我们这里将会对 ModelSim 软件中几个常用小工具进行简单的讲解，如图 2-11 所示。

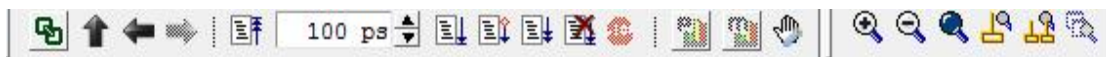


图 2-11

前面的几个放大镜模样的工具是放大、缩小等功能，鼠标放到图标上会显示出它们的快捷键，中间的黄色图标是用来在波形图上做标记的，后面的则是用来运行仿真，暂停仿真、重新运行等功能，大家可以自行尝试。如图 2-12 所示界面，就是我们利用上述工具制作出来的。

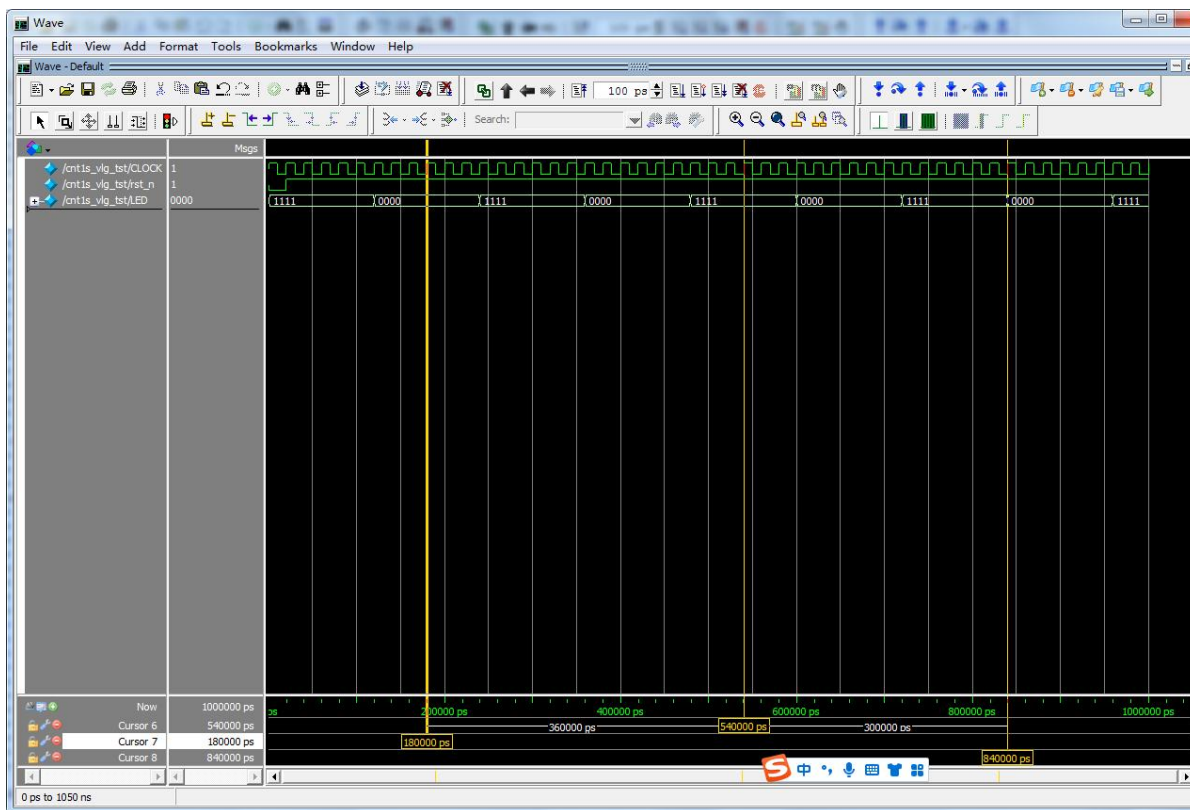


图 2-12

在TestBench 代码中 CLOCK 的初值为低电平，周期 20ns 为周期，rst_n 在第一个周期内为低电平，以后的周期中变为高电平。从仿真图中可以看出，CLOCK 和rst_n 的波形与代码一致。再观察仿真图

中的 `cnt_value`, `rst_n` 置高之后, 随后的第一个 `CLOCK` 上升沿到来时 `cnt_value` 的值即加 1, 一直到 `cnt_value` 的值等于 5, 当下一个 `CLOCK` 的上升沿到来时, `cnt_value` 的值归零, 同时 `LED` 的值发生反转, 与我们的 Verilog 代码所预期的一致。

这里我们需要注意的是: 我们修改 Verilog 程序代码和仿真文件代码 (前提是不修改端口信号), 我们是可以不需要重新编译工程的, 我们只要关闭 ModelSim 仿真软件, 再一次点击开始仿真即可。如果我们修改了端口的输入和输出, 那么这里最好重新全编译一次, 并且我们需要重新生成仿真模板文件进行修改。至此, 我们就完成了功能仿真。